

Selenium & TestNG Advanced Project Assessment

- **Level:** Advanced
- **AUT (Application Under Test):** <https://the-internet.herokuapp.com>
- **Login Credentials:** tomsmith / SuperSecretPassword!

1. What Is Already Given (Do NOT Rewrite)

The provided project template fully implements the following components (assume they work correctly):

- **All import statements:** For Selenium, TestNG, ExtentReports, Log4J2, etc.
- **All constants:** BASE_URL, VALID_USER, VALID_PASS, TIMEOUT, and various directory paths.
- **Logger log:** Pre-configured Log4J2 logger instance.
- **ExtentReports extent:** Pre-configured static reporter instance shared across tests.
 - Uses a thread-local structure (ThreadLocal<ExtentTest>) for per-thread report nodes.
- **Thread-safe WebDriver management:**
 - Managed via ThreadLocal<WebDriver> driverTL and ThreadLocal<WebDriverWait> waitTL.
 - Accessible via convenience methods: getDriver() and getWait().
- **Convenience accessors & Lifecycle methods:**
 - @BeforeSuite initReporting(): Creates output folders, initializes ExtentSparkReporter (using the DARK theme), and records global system info.
 - @AfterSuite flushReporting(): Explicitly calls extent.flush() to generate the report.
 - captureScreenshot(String name): Saves a .png snapshot to the test-reports/screenshots/ directory and returns its string file path.
 - step(String msg): Simultaneously writes a log entry to both Log4J2 and the active ExtentReport node.

- `sleep(long ms)`: Syntactic wrapper around `Thread.sleep` that cleanly catches and handles `InterruptedException`.

2. What You Must Implement — 6 TODOs

TODO 1 — Page Object Classes (4 Static Inner Classes)

You must write each of these pages as a public static inner class inside your main `SeleniumTestSuite` file.

A. `LoginPage` — Maps to path `/login`

- **Locators (use exactly the strategy shown):**
 - `USERNAME` $\rightarrow$ `By.id("username")`
 - `PASSWORD` $\rightarrow$ `By.id("password")`
 - `LOGIN_BTN` $\rightarrow$ `By.cssSelector("button[type='submit']")`
 - `SUCCESS` $\rightarrow$ `By.cssSelector(".flash.success")`
 - `ERROR` $\rightarrow$ `By.cssSelector(".flash.error")`
 - `HEADING` $\rightarrow$ `By.tagName("h2")`
- **Constructor:** `LoginPage(WebDriver driver)` $\rightarrow$ Executes `driver.get(BASE_URL + "/login")`.
- **Methods:**
 - `enterUsername(String u)`: Returns `LoginPage`. Clears field, then runs `sendKeys(u)`.
 - `enterPassword(String p)`: Returns `LoginPage`. Clears field, then runs `sendKeys(p)`.
 - `clickLogin()`: Returns `void`. Clicks the `LOGIN_BTN`.
 - `getSuccessMessage()`: Returns `String`. Extracts `getText().trim()` from the `SUCCESS` element.
 - `getErrorMessage()`: Returns `String`. Extracts `getText().trim()` from the `ERROR` element.
 - `getPageHeading()`: Returns `String`. Extracts `getText().trim()` from the `HEADING` element.

- `isUsernameDisplayed()`: Returns boolean. Checks `isDisplayed()` on the USERNAME field.

B. DropdownPage — Maps to path /dropdown

- **Locator:** `DROPDOWN` $\rightarrow$ `By.id("dropdown")`
- **Constructor:** `DropdownPage(WebDriver driver)` $\rightarrow$ Navigates directly to the /dropdown path.
- **Private Helper:** `Select select()` $\rightarrow$ Returns a new `Select(driver.findElement(DROPDOWN))`.
- **Methods:**
 - `selectByVisibleText(String text)` $\rightarrow$ Invokes `select().selectByVisibleText(text)`.
 - `selectByValue(String val)` $\rightarrow$ Invokes `select().selectByValue(val)`.
 - `selectByIndex(int idx)` $\rightarrow$ Invokes `select().selectByIndex(idx)`.
 - `getSelectedText()` $\rightarrow$ Returns a String by executing `select().getFirstSelectedOption().getText()`.

C. AlertsPage — Maps to path /javascript_alerts

- **Locators (Use `By.xpath` with a `text()` predicate for buttons):**
 - `BTN_ALERT` $\rightarrow$ `By.xpath("//button[text()='Click for JS Alert']")`
 - `BTN_CONFIRM` $\rightarrow$ `By.xpath("//button[text()='Click for JS Confirm']")`
 - `BTN_PROMPT` $\rightarrow$ `By.xpath("//button[text()='Click for JS Prompt']")`
 - `RESULT` $\rightarrow$ `By.id("result")`
- **Constructor:** `AlertsPage(WebDriver driver)` $\rightarrow$ Navigates directly to the /javascript_alerts path.
- **Methods:**
 - `triggerAlert()` / `triggerConfirm()` / `triggerPrompt()` $\rightarrow$ Clicks its respective target button.
 - `getAlertText()` $\rightarrow$ Returns a String via `driver.switchTo().alert().getText()`.

- `acceptAlert()` $\rightarrow$ Executes `driver.switchTo().alert().accept()`.
- `dismissAlert()` $\rightarrow$ Executes `driver.switchTo().alert().dismiss()`.
- `sendKeysAndAccept(String t)` $\rightarrow$ Sends keys to the alert context, then immediately accepts it.
- `getResult()` $\rightarrow$ Returns a String using `getText().trim()` on the RESULT element.

D. DynamicPage — Maps to path `/dynamic_loading/{example}`

- **Locators:**

- START $\rightarrow$ `By.cssSelector("#start button")`
- FINISH $\rightarrow$ `By.cssSelector("#finish h4")`

- **Constructor:** `DynamicPage(WebDriver driver, int example)` $\rightarrow$ Navigates directly to `BASE_URL + "/dynamic_loading/" + example`.

- **Methods:**

- `clickStart()` $\rightarrow$ Clicks the designated START button.
- `waitExplicit(WebDriverWait wait)` $\rightarrow$ Evaluates `wait.until(ExpectedConditions.visibilityOfElementLocated(FINISH))`, then returns its trimmed text value.
- `waitFluent()` $\rightarrow$ Initializes a new `FluentWait<>(driver)` configured with a `timeout=20s`, `polling=500ms`, ignoring `NoSuchElementException`. Waits for the finish element to display, and returns its trimmed text string.

TODO 2 — @BeforeMethod: Browser Setup

Implement a driver lifecycle setup method matching this signature:

Java

`@Parameters({"browser"})`

`@BeforeMethod(alwaysRun = true)`

`public void setupBrowser(@Optional("chrome") String browser, Method method)`

- **Logging Execution:** Print a structured log line pattern: $\blacktriangleright$ `[BROWSER] - methodName`.

- **Firefox Setup Branch:** If the browser parameters match "firefox", call `WebDriverManager.firefoxdriver().setup()` and initialize a new `FirefoxDriver(opts)` with the `--headless` argument active.
- **Chrome Setup Branch (Default fallback):** Run `WebDriverManager.chromedriver().setup()` and open a new `ChromeDriver(opts)` running with the following initialization parameters:
 - `--headless=new`
 - `--no-sandbox`
 - `--disable-dev-shm-usage`
 - `--window-size=1920,1080`
- **Timeouts & Window Rules:**
 - Configure an implicit wait time block of 5 seconds via `d.manage().timeouts().implicitlyWait(Duration.ofSeconds(5))`.
 - Call `d.manage().window().maximize()`.
 - Bind the driver to thread-safe storage: `driverTL.set(d)` and initialize `waitTL.set(new WebDriverWait(d, Duration.ofSeconds(TIMEOUT)))`.
- **Reporting Hooks:** Extract the `@Test` annotation context dynamically from the executed test method parameter. Fallback to the raw method string name if a custom test description field is absent. Create the test node via:

```
extentTest.set(extent.createTest "[" + browser + "]" - " + method.getName(), description))
```

TODO 3 — @AfterMethod: Teardown + Screenshot on Failure

Implement a teardown lifecycle method matching this signature:

Java

```
@AfterMethod(alwaysRun = true)
```

```
public void teardownBrowser(ITestResult result)
```

- **State Assessment Logic:** Read the incoming execution status via `result.getStatus()` and react accordingly:

- **ITestResult.FAILURE:** Log an explicit error `log.error(" ❌ FAILED")`. Capture the application state using `captureScreenshot(name)` and attach it within a safe try-catch wrapper:

```
extentTest.get().fail(result.getThrowable()).addScreenCaptureFromPath(path)
```

- **ITestResult.SKIP:** Write a warning message `log.warn(" ⚠️ SKIPPED")` and pass skipped flag markers downward to `extentTest.get().skip(...)`.
- **SUCCESS (Default/Fallback condition):** Log `log.info(" ✅ PASSED")` and flag your reporter node using `extentTest.get().pass("Test Passed")`.
- **Global Cleanup Rule:** Wrap a safe execution block that checks if the driver instance is active. If true, run `driverTL.get().quit()`. Finally, clear out the current thread allocations completely using `driverTL.remove()` and `waitTL.remove()`.

TODO 4 — @DataProvider

Implement a parallel-disabled TestNG data feeding method:

Java

```
@DataProvider(name = "loginData", parallel = false)
```

```
public Object[][] loginData()
```

It must deliver exactly **4 rows** containing the dataset elements: (username, password, expectedOutcome, scenarioLabel).

Row Index	Username Param	Password Param	Expected Outcome Value	Scenario Label Description
1	"tomsmith"	"SuperSecretPassword!"	"success"	"Valid credentials"
2	"tomsmith"	"wrongpassword"	"failure"	"Wrong password"
3	"invaliduser"	"SuperSecretPassword!"	"failure"	"Wrong username"

Row Index	Username Param	Password Param	Expected Outcome Value	Scenario Label Description
4	"" (<i>Empty String</i>)	"password11"	"failure"	"Empty credentials"

TODO 5 — Test Cases (8 Methods)

Crucial Rule: Every test case method listed below must explicitly explicitly pass groups, description, and priority parameters inside its active @Test marker layout.

- **TC001 — Valid Login** | groups={"smoke", "regression"} | priority=1
 - Instantiate a new LoginPage element, and run a chain sequence: `enterUsername(VALID_USER).enterPassword(VALID_PASS).clickLogin()`.
 - Establish an explicit wait expecting the flash success toast alert element to reveal.
 - Assert validation parameters via `Assert.assertTrue(successMessage.contains("You logged into a secure area!"))`.
 - Confirm that `Assert.assertTrue(currentUrl.endsWith("/secure"))` returns valid.
- **TC002 — Invalid Login** | groups={"smoke", "negative"} | priority=2
 - Instantiate a new LoginPage sequence, passing incorrect string parameters: `enterUsername("badUser").enterPassword("badPass").clickLogin()`.
 - Run an explicit wait expecting the error flash container element to emerge.
 - Assert validation parameters via `Assert.assertTrue(errorMessage.contains("Your username is invalid!"))`.
- **TC003 — SoftAssert Example** | groups={"smoke"} | priority=3
 - Build a standard LoginPage layout instance and declare a local verification controller: `SoftAssert soft = new SoftAssert()`.
 - Queue structural page checks safely using soft validation sequences:
 - `soft.assertTrue(title.contains("The Internet"));`

- `soft.assertEquals(getPageHeading(), "Login Page");`
 - `soft.assertTrue(currentUrl.contains("login"));`
 - `soft.assertTrue(isUsernameDisplayed());`
 - Finalize validation routines on the final line by calling `soft.assertAll()`.
- **TC004 — Dropdown Verification** | groups={"smoke", "regression"} | priority=4
 - Initialize an isolated instance targeting the DropdownPage locator layout.
 - Trigger method validation sequences sequentially:
 - Run `selectByVisibleText("Option 1")` $\rightarrow$ Validate selected text matches "Option 1".
 - Run `selectByValue("2")` $\rightarrow$ Validate selected text matches "Option 2".
 - Run `selectByIndex(1)` $\rightarrow$ Validate selected text matches "Option 1".
- **TC005 — Alert Handling (All 3 variations)** | groups={"regression"} | priority=5
 - **Part A — Accept Routine:** Call `triggerAlert()`, run wait validations $\rightarrow$ verify string content matches "I am a JS Alert". Execute `accept()`, then verify output results update to match "You successfully clicked an alert".
 - **Part B — Dismiss Routine:** Call `triggerConfirm()`, run wait validations $\rightarrow$ execute `dismiss()`, then verify output text reports "You clicked: Cancel".
 - **Part C — Prompt Input Routine:** Call `triggerPrompt()`, run wait validations $\rightarrow$ execute an input write operation via `sendKeysAndAccept("Hello TestNG")`. Confirm that output text properties contain "You entered: Hello TestNG".
- **TC006 — Multiple Windows Execution** | groups={"regression"} | priority=6
 - *Design Constraint:* Write this step natively inside the method layer context directly—**do not use a separate Page Object Model framework pattern.**

- Direct your browser instance to load `BASE_URL + "/windows"`, capturing a baseline context handle anchor reference variable: `String original = getWindowHandle()`.
 - Click on target link `By.linkText("Click Here")` and build an execution pause loop matching `ExpectedConditions.numberOfWindowsToBe(2)`.
 - Parse active windows to extract the newly spawned tab target reference handle (filtering out the value matching your original layout variable).
 - Run `switchTo().window(newHandle)` $\rightarrow$ check if active application title reads "New Window".
 - Finalize interaction parameters by processing `close()`, returning back via `switchTo().window(original)`, and verifying active tab counters decrement smoothly back down to 1.
- **TC007 — Wait Strategies Assessment** | groups={"regression", "waits"} | priority=7
 - **Part A — Explicit Wait Approach:** Instantiate a new `DynamicPage(getDriver(), 1)` runtime path. Execute `clickStart()`, introduce a structural pause trace via `sleep(300)`, and call `waitExplicit(getWait())`. Assert that the returned text matches "Hello World!".
 - **Part B — Fluent Wait Approach:** Navigate directly to the alternate resource path `/dynamic_loading/2`. Instantiate a separate page sequence targeting new `DynamicPage(getDriver(), 2)`. Invoke `clickStart()`, immediately pass execution control over to `waitFluent()`, and assert that the returned text evaluation returns "Hello World!".
 - **TC008 — Data-Driven Login Loop** | groups={"regression", "data-driven"} | dataProvider="loginData" | priority=8
 - Configure your active testing method frame signature to receive parameters smoothly: `public void testDataDrivenLogin(String username, String password, String expected, String scenario)`
 - Call `step(scenario)` to register the test case iteration cleanly into your reporting pipeline logs.
 - Initialize an interface handling your `LoginPage` layout elements, passing individual data matrix inputs: `enterUsername(username).enterPassword(password).clickLogin()`.

- Evaluation Routing logic check: If the passed expected tracking string value reads "success", introduce explicit pauses until URLs reflect containing /secure, and finalize standard assertions. If the variable evaluation yields a status of "failure", wait for flash errors to load and verify the return string value does not arrive empty.

TODO 6 — Programmatic TestNG Execution (main method)

Write a standard executable bootstrap entry path method pattern within the codebase configuration to run your suites locally without relying on external workspace XML engine tools:

Java

```
public static void main(String[] args)
```

- Initialize a programmatic TestNG suite instance mapping matching variable identifiers: `XmlSuite suite = new XmlSuite();`. Set its runtime tag identifier via `suite.setName("The Internet Suite");`, setting parallel parameters to `XmlSuite.ParallelMode.NONE`, and setting thread pool execution capacities using `suite.setThreadCount(2);`.
- Inject global application execution testing environment parameter values into the run context using: `suite.setDataProviderThreadCount(2);` or `suite.setParameters(Map.of("browser", "chrome"));`.
- Instantiate an suite mapping block trace: `XmlTest test = new XmlTest(suite);` `test.setName("Full Regression");`. Add your automation execution test reference blueprint source map file path layout tracking: `test.setXmlClasses(List.of(new XmlClass(SeleniumTestSuite.class)));`.
- *Development Directive Code Snippet Instruction:* Ensure your codebase contains a commented line indicating how a user would isolate executions strictly down to a specific target group cluster if necessary:

Java

```
// test.setIncludedGroups(Collections.singletonList("smoke"));
```

- Run execution tasks sequentially: Initialize an instance using TestNG `tng = new TestNG();`, register the custom program target via `tng.setXmlSuites(List.of(suite));`, and execute the test pipeline using `tng.run();`.

- Print a structural console message layout notifying tracking logs when executions conclude successfully: `System.out.println("Reports generated in test-reports/");`.

3. Expected Console & Document Tree Structure Output

Upon standard test suites reaching complete run finalization passes, reports write down to target project output directories organized sequentially below:

Plaintext

test-reports/

├── Report_20241201_130000.html <-- Main execution ExtentReport dashboard layout

└── screenshots/

 └── TC00X_SomeTest_20241201_130045.png <-- Automatically saved step crash captures

4. Evaluation Criteria Matrix Weights

Submitted project responses measure against individual architectural implementation guidelines matching parameters below:

Evaluation Criteria Category Area	Assessment Weight Target Metric
Ui/Ux Layout Presentation Check	15%
CodeQuality & Standard Architecture Formatting Patterns	25%
System Testing Structure Design & Robust Locator Selection Choices	25%
Feature Functionality Operational Correctness Execution Metrics	30%
DatabaseDesign Configuration Parameters	5%

```
package backend_api;

import com.aventstack.extentreports.*;
import com.aventstack.extentreports.reporter.*;
import com.aventstack.extentreports.reporter.configuration.Theme;
import io.github.bonigarcia.wdm.WebDriverManager;
import org.apache.logging.log4j.*;
import org.openqa.selenium.*;
import org.openqa.selenium.chrome.*;
import org.openqa.selenium.firefox.*;
import org.openqa.selenium.interactions.Actions;
import org.openqa.selenium.support.ui.*;
import org.testng.*;
import org.testng.annotations.*;
import org.testng.asserts.SoftAssert;
import org.testng.xml.*;

import java.io.*;
import java.lang.reflect.Method;
import java.nio.file.*;
import java.time.*;
import java.time.format.DateTimeFormatter;
import java.util.*;
```

```
/**
```

```
 * seleniumTestSuite.java - STUDENT TEMPLATE
```

* AUT : <https://the-internet.herokuapp.com>

* Time : 2.5 Hours | Submit this single .java file only

* * Compile : mvn test-compile

* Run : mvn test OR run main() directly

*/

```
public class SeleniumTestSuite {
```

```
    // == CONSTANTS (given) ==
```

```
    private static final String BASE_URL = "https://the-internet.herokuapp.com";
```

```
    private static final String VALID_USER = "tomsmith";
```

```
    private static final String VALID_PASS = "SuperSecretPassword!";
```

```
    private static final int TIMEOUT = 15;
```

```
    private static final String REPORTS_DIR = "test-reports";
```

```
    private static final String SCREENSHOTS_DIR = REPORTS_DIR + "/screenshots";
```

```
    // == LOGGING (given) ==
```

```
    private static final Logger log = LogManager.getLogger(SeleniumTestSuite.class);
```

```
    // == EXTENT REPORTS (given) ==
```

```
    private static ExtentReports extent = null;
```

```
    private static final ThreadLocal<ExtentTest> extentTest = new ThreadLocal<>();
```

```
    // == WEBDRIVER - ThreadLocal (given) ==
```

```
    private static final ThreadLocal<WebDriver> driverTL = new ThreadLocal<>();
```

```
    private static final ThreadLocal<WebDriverWait> waitTL = new ThreadLocal<>();
```

```

public static WebDriver getDriver() { return driverTL.get(); } // given
public static WebDriverWait getWait() { return waitTL.get(); } // given

//
=====

// TODO 1 - PAGE OBJECT MODEL (4 static inner classes)

//
=====

/**
 * TODO 1a: LoginPage (URL: BASE_URL + "/login")
 * Locators (use the exact strategy shown):
 * USERNAME -> By.id("username")
 * PASSWORD -> By.id("password")
 * LOGIN_BTN -> By.cssSelector("button[type='submit']")
 * SUCCESS -> By.cssSelector(".flash.success")
 * ERROR -> By.cssSelector(".flash.error")
 * HEADING -> By.tagName("h2")
 *
 * Constructor: takes WebDriver, calls driver.get(BASE_URL + "/login")
 *
 * Methods (enterUsername / enterPassword return 'this' for chaining):
 * LoginPage enterUsername(String u) -> clear + sendKeys
 * LoginPage enterPassword(String p) -> clear + sendKeys
 * void clickLogin()
 * String getSuccessMessage() -> getText().trim() on SUCCESS
 * String getErrorMessage() -> getText().trim() on ERROR

```

```

* String getPageHeading() -> getText().trim() on HEADING
* boolean isUsernameDisplayed() -> isDisplayed() on USERNAME
*/

public static class LoginPage {

    private final WebDriver driver;

    public LoginPage(WebDriver driver) {

        this.driver = driver;

    }

    // TODO 1a: declare By locators, constructor, and all 7 methods
}

/**
* TODO 1b: DropdownPage (URL: BASE_URL + "/dropdown")
* Locator: DROPDOWN -> By.id("dropdown")
* Constructor: takes WebDriver, navigates to /dropdown
* Private helper: Select select() -> new Select(driver.findElement(DROPDOWN))
*
* Methods:
* void selectByVisibleText(String text)
* void selectByValue(String val)
* void selectByIndex(int idx)
* String getSelectedText() -> select().getFirstSelectedOption().getText()
*/

public static class DropdownPage {

```

```

private final WebDriver driver;

public DropdownPage(WebDriver driver) {
    this.driver = driver;
}

// TODO 1b
}

/**
 * TODO 1c: AlertsPage (URL: BASE_URL + "/javascript_alerts")
 * Locators (all By.xpath with text() predicate except RESULT):
 * BTN_ALERT -> By.xpath("//button[text()='Click for JS Alert']")
 * BTN_CONFIRM -> By.xpath("//button[text()='Click for JS Confirm']")
 * BTN_PROMPT -> By.xpath("//button[text()='Click for JS Prompt']")
 * RESULT -> By.id("result")
 *
 * Constructor: takes WebDriver, navigates to /javascript_alerts
 *
 * Methods:
 * void triggerAlert() / triggerConfirm() / triggerPrompt()
 * String getAlertText() -> driver.switchTo().alert().getText()
 * void acceptAlert() -> driver.switchTo().alert().accept()
 * void dismissAlert() -> driver.switchTo().alert().dismiss()
 * void sendKeysAndAccept(String t) -> get alert, sendKeys(t), accept()
 * String getResult() -> getText().trim() on RESULT

```

```

*/

public static class AlertsPage {

    private final WebDriver driver;

    public AlertsPage(WebDriver driver) {

        this.driver = driver;

    }

    // TODO 1c

}

/**
 * TODO 1d: DynamicPage (URL: BASE_URL + "/dynamic_loading/{example}")
 * Locators:
 * START -> By.cssSelector("#start button")
 * FINISH -> By.cssSelector("#finish h4")
 *
 * Constructor: takes WebDriver + int example, navigates to /dynamic_loading/<example>
 *
 * Methods:
 * void clickStart()
 * String waitExplicit(WebDriverWait wait)
 * -> wait.until(ExpectedConditions.visibilityOfElementLocated(FINISH))
 * -> return FINISH element getText().trim()
 * String waitFluent()
 * -> new FluentWait<>(driver)

```

```

* .withTimeout(Duration.ofSeconds(20))
* .pollingEvery(Duration.ofMillis(500))
* .ignoring(NoSuchElementException.class)
* .until(d -> d.findElement(FINISH))
* .getText().trim()
*/

public static class DynamicPage {
    private final WebDriver driver;

    public DynamicPage(WebDriver driver, int example) {
        this.driver = driver;
    }

    // TODO 1d
}

//
=====

// == EXTENT REPORT LIFECYCLE - GIVEN (do not modify) ==

//
=====

@BeforeSuite(alwaysRun = true)
public void initReporting() {
    new File(REPORTS_DIR).mkdirs();
    new File(SCREENSHOTS_DIR).mkdirs();

    String ts =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_HH:mm:ss"));

```

```
String path = REPORTS_DIR + "/Report_" + ts + ".html";
ExtentSparkReporter spark = new ExtentSparkReporter(path);
spark.config().setDocumentTitle("Selenium TestNG Report");
spark.config().setReportName("The Internet - Automation Suite");
spark.config().setTheme(Theme.DARK);
extent = new ExtentReports();
extent.attachReporter(spark);
extent.setSystemInfo("AUT", BASE_URL);
extent.setSystemInfo("OS", System.getProperty("os.name"));
log.info("ExtentReports ready -> {}", path);
}
```

```
@AfterSuite(alwaysRun = true)
public void flushReporting() {
    if (extent != null) extent.flush();
    log.info("ExtentReports flushed.");
}
```

```
//
```

```
=====
```

```
// == Utilities - GIVEN (do not modify) ==
```

```
//
```

```
=====
```

```
private String captureScreenshot(String name) {
    try {
        File src = ((TakesScreenshot) getDriver()).getScreenshotAs(OutputType.FILE);
```

```
        String ts =
LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyyMMdd_HHmmss"));

        String dest = SCREENSHOTS_DIR + "/" + name + "_" + ts + ".png";

        Files.copy(src.toPath(), Paths.get(dest));

        log.info("Screenshot -> {}", dest);

        return dest;

    } catch (Exception e) {

        log.error("Screenshot failed: {}", e.getMessage());

        return null;

    }

}
```

```
private void step(String msg) {

    log.info("STEP: {}", msg);

    extentTest.get().info(msg);

}
```

```
private void sleep(long ms) {

    try { Thread.sleep(ms); } catch (InterruptedException e) {
Thread.currentThread().interrupt(); }

}
```

```
//
```

```
=====
```

```
// TODO 2 - @BeforeMethod: Browser Setup [15 marks]
```

```
//
```

```
=====
```

```

/**
 * * Annotate with @Parameters("browser") and @BeforeMethod(alwaysRun = true)
 * * Signature: public void setupBrowser(@Optional("chrome") String browser, Method
method)
 * * Steps:
 * 1. Log: "► [BROWSER] -> methodName"
 * 2. If "firefox":
 * WebDriverManager.firefoxdriver().setup()
 * new FirefoxDriver(new FirefoxOptions().addArguments("--headless"))
 * Else (chrome default):
 * WebDriverManager.chromedriver().setup()
 * ChromeOptions with: "--headless=new", "--no-sandbox",
 * "--disable-dev-shm-usage", "--window-size=1920,1080"
 * new ChromeDriver(options)
 * 3. d.manage().timeouts().implicitlyWait(Duration.ofSeconds(5))
 * 4. d.manage().window().maximize()
 * 5. driverTL.set(d)
 * 6. waitTL.set(new WebDriverWait(d, Duration.ofSeconds(TIMEOUT)))
 * 7. Read @Test annotation from Method, get description (fallback to method name)
 * extentTest.set(extent.createTest "[" + BROWSER + "]" methodName, description))
 */
// TODO 2: write the @BeforeMethod here

```

```

//
=====

// TODO 3 - @AfterMethod: Teardown + Screenshot on Failure [10 marks]

```

```
//
```

```
=====
```

```
/**
```

```
 * * Annotate with @AfterMethod(alwaysRun = true)
```

```
 * * Signature: public void teardownBrowser(ITestResult result)
```

```
 * * * Use result.getStatus() (ITestResult.FAILURE / SKIP / else SUCCESS):
```

```
 * * FAILURE:
```

```
 * - log.error(" ❌ FAILED: name")
```

```
 * - String ss = captureScreenshot(result.getName())
```

```
 * - extentTest.get().fail(result.getThrowable())
```

```
 * - try: extentTest.get().addScreenCaptureFromPath(ss, "Failure Screenshot")
```

```
 * * SKIP:
```

```
 * - log.warn(" ⚠️ SKIPPED: name")
```

```
 * - extentTest.get().skip("Test Skipped")
```

```
 * * SUCCESS (default):
```

```
 * - log.info(" ✅ PASSED: name")
```

```
 * - extentTest.get().pass("Test Passed")
```

```
 * * * Always (after the switch):
```

```
 * - WebDriver d = driverTL.get()
```

```
 * - if (d != null) -> d.quit(), driverTL.remove(), waitTL.remove()
```

```
 */
```

```
// TODO 3: write the @AfterMethod here
```

```
//
```

```
=====
```

```
// TODO 4 - @DataProvider [5 marks]
```

```
//
```

```
=====
```

```
/**
```

```
 * * @DataProvider(name = "loginData", parallel = false)
```

```
 * * Returns Object[][] with exactly 4 rows.
```

```
 * * Each row: { username, password, expectedOutcome, scenarioLabel }
```

```
 * * * Row 1: { "tomsmith", "SuperSecretPassword!", "success", "Valid credentials" }
```

```
 * * Row 2: { "tomsmith", "wrongpassword", "failure", "Wrong password" }
```

```
 * * Row 3: { "invaliduser", "SuperSecretPassword!", "failure", "Wrong username" }
```

```
 * * Row 4: { "", "password11", "failure", "Empty credentials" }
```

```
 */
```

```
// TODO 4: write the @DataProvider method here
```

```
//
```

```
=====
```

```
// TODO 5 - TEST CASES (8 methods) [50 marks - 6-7 marks each]
```

```
//
```

```
=====
```

```
/**
```

```
 * TC001 - Valid Login
```

```
 * @Test: groups={"smoke", "regression"}, description="TC001: Valid login redirects to  
/secure", priority=1
```

```
 * * Steps:
```

```
 * 1. step("Enter valid credentials and log in")
```

```

* 2. Create LoginPage ->
enterUsername(VALID_USER).enterPassword(VALID_PASS).clickLogin()

* 3. getWait().until(visibilityOfElementLocated(By.cssSelector(".flash.success")))

* 4. Assert: getSuccessMessage() contains "You logged into a secure area!"

* 5. Assert: getCurrentUrl() ends with "/secure"

* 6. extentTest.get().pass("Login successful.")

*/

// TODO 5a: TC001_ValidLogin

/**

* TC002 - Invalid Login

* @Test: groups={"smoke", "negative"}, description="TC002: Invalid login shows error
flash", priority=2

* * Steps:

* 1. Create LoginPage ->
enterUsername("badUser").enterPassword("badPass").clickLogin()

* 2. getWait().until(visibilityOfElementLocated(By.cssSelector(".flash.error")))

* 3. Assert: getErrorMessage() contains "Your username is invalid!"

*/

// TODO 5b: TC002_InvalidLogin

/**

* TC003 - SoftAssert: title, heading, URL, field visibility

* @Test: groups={"smoke"}, description="TC003: SoftAssert on page attributes",
priority=3

```

* * Steps:

* 1. Create LoginPage (navigates automatically)

* 2. SoftAssert soft = new SoftAssert()

* 3. soft.assertTrue(title contains "The Internet")

* 4. soft.assertEquals(getPageHeading(), "Login Page")

* 5. soft.assertTrue(URL contains "login")

* 6. soft.assertTrue(isUsernameDisplayed())

* 7. soft.assertAll() <- MUST be last

*/

// TODO 5c: TC003_SoftAssertions

/**

* TC004 - Dropdown: all 3 selection methods

* @Test: groups={"smoke", "regression"}, description="TC004: Verify all 3 Select strategies", priority=4

* * Steps:

* 1. Create DropdownPage

* 2. selectByVisibleText("Option 1") -> Assert.assertEquals(getSelectedText(), "Option 1")

* 3. selectByValue("2") -> Assert.assertEquals(getSelectedText(), "Option 2")

* 4. selectByIndex(1) -> Assert.assertEquals(getSelectedText(), "Option 1")

*/

// TODO 5d: TC004_DropdownSelection

/**

* TC005 - JS Alert: Accept, Confirm Dismiss, Prompt Input (all 3 in one test)

* @Test: groups={"regression"}, description="TC005: Accept alert, dismiss confirm, send keys to prompt", priority=5

* * Part A - JS Alert Accept:

* triggerAlert() -> wait alertIsPresent() -> assertEquals(getAlertText(), "I am a JS Alert")

* acceptAlert() -> assertEquals(getResult(), "You successfully clicked an alert")

* Part B - JS Confirm Dismiss:

* triggerConfirm() -> wait alertIsPresent() -> dismissAlert()

* assertEquals(getResult(), "You clicked: Cancel")

* Part C - JS Prompt:

* triggerPrompt() -> wait alertIsPresent() -> sendKeysAndAccept("Hello TestNG")

* assertEquals(getResult(), "You entered: Hello TestNG")

*/

// TODO 5e: TC005_AlertHandling

/**

* TC006 - Multiple Windows

* @Test: groups={"regression"}, description="TC006: Open new window, verify title, close and return", priority=6

* * Note: Write this test INLINE - no Page Object needed.

* Steps:

* 1. getDriver().get(BASE_URL + "/windows")

* 2. String original = getDriver().getWindowHandle()

* 3. getDriver().findElement(By.linkText("Click Here")).click()

* 4. getWait().until(numberOfWindowsToBe(2))

* 5. Find new handle: getDriver().getWindowHandles(), filter != original, findFirst()

```

* 6. switchTo().window(newHandle)

* 7. Assert.assertEquals(getDriver().getTitle(), "New Window")

* 8. getDriver().close()

* 9. switchTo().window(original)

* 10. Assert.assertEquals(getDriver().getWindowHandles().size(), 1)

*/

// TODO 5f: TC006_MultipleWindows


/**

* TC007 - Wait Strategies: Explicit Wait + Fluent Wait (+ Thread.sleep demo)

* @Test: groups={"regression", "waits"}, description="TC007: Explicit Wait and Fluent
Wait demo", priority=7

* * Part A - Explicit Wait (Example 1):

* DynamicPage dp1 = new DynamicPage(getDriver(), 1)

* dp1.clickStart()

* sleep(300) <- Thread.sleep for demonstration contrast

* Assert.assertEquals(dp1.waitExplicit(getWait()), "Hello World!")

* Part B - Fluent Wait (Example 2):

* Navigate to BASE_URL + "/dynamic_loading/2"

* DynamicPage dp2 = new DynamicPage(getDriver(), 2)

* dp2.clickStart()

* Assert.assertEquals(dp2.waitFluent(), "Hello World!")

*/

// TODO 5g: TC007_WaitStrategies

```

```

/**
 * TC008 - Data-Driven Login using @DataProvider
 *
 * @Test: groups={"regression", "data-driven"}, description="TC008: Data-driven login - 4
scenarios",
 *
 * dataProvider="loginData", priority=8
 *
 * Signature MUST include all 4 DataProvider params:
 *
 * (String username, String password, String expected, String scenario)
 *
 * * Steps:
 *
 * 1. step("Scenario: [" + scenario + "] | User: " + username)
 *
 * 2. extentTest.get().info("Scenario: " + scenario)
 *
 * 3. Create LoginPage ->
enterUsername(username).enterPassword(password).clickLogin()
 *
 * 4. if expected.equals("success"):
 *
 * Wait until URL contains "/secure"
 *
 * Assert URL contains "/secure"
 *
 * Else:
 *
 * Wait until ".flash.error" is visible
 *
 * Assert error text is not empty
 */
// TODO 5h: TC008_DataDrivenLogin

//
=====

// TODO 6 - main(): Programmatic TestNG Execution [10 marks]

```

```
//
=====

/**
 * * Build and run TestNG without an external testng.xml.
 * * * Steps:
 * 1. XmlSuite suite = new XmlSuite()
 * suite.setName("The Internet - Selenium Suite")
 * suite.setParallel(XmlSuite.ParallelMode.NONE)
 * suite.setThreadCount(2)
 * suite.setParameters(Collections.singletonMap("browser", "chrome"))
 * * 2. XmlTest xmlTest = new XmlTest(suite)
 * xmlTest.setName("Full Regression")
 * xmlTest.setClasses(Collections.singletonList(
 * new XmlClass(SeleniumTestSuite.class.getName()))
 * * // Bonus: add this line (commented out) to run only smoke:
 * // xmlTest.setIncludedGroups(Collections.singletonList("smoke"));
 * * 3. TestNG runner = new TestNG()
 * runner.setXmlSuites(Collections.singletonList(suite))
 * runner.run()
 * * 4. System.out.println("Reports -> " + REPORTS_DIR + "/")
 */
public static void main(String[] args) {
    // TODO 6
}
}
```